

vGPU Features

Multi-vGPU

Multi-vGPU technology enables a single virtual machine (VM) to utilize multiple vGPUs simultaneously, significantly enhancing its computational capabilities. Unlike the standard vGPU setup, where one physical GPU is virtualized and shared across multiple VMs, multi-vGPU allows a single VM to aggregate resources from multiple vGPU devices that do not need to reside on the same physical GPU; they can be distributed across separate GPUs, pooling their power to meet the demands of high-performance workloads.

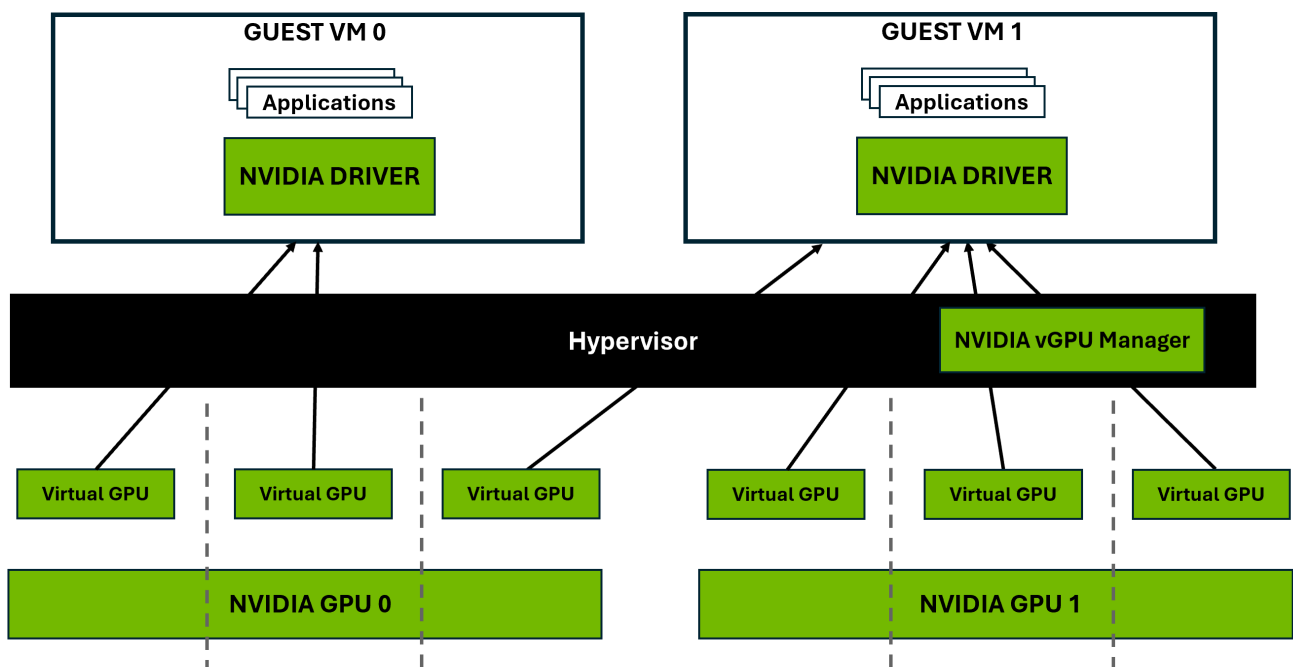


Figure 1: Example of Multi-vGPU Setup

Example of Multi-vGPU Setup

Figure 1 illustrates a multi-vGPU setup, where physical NVIDIA GPUs are divided into multiple vGPUs that are assigned to VMs. Guest VM 0 has all its vGPUs allocated from a single physical GPU, while Guest VM 1 has its vGPUs distributed across multiple physical GPUs. This flexibility allows VMs to access GPU resources based on workload demands, enabling efficient GPU utilization and scalability in virtualized environments.

This feature is particularly advantageous for workloads that require extensive computational power, such as:

- **Optimized Resource Allocation:** Multi-vGPU allows applications within a VM to get their own dedicated GPU resources. For instance, a VM (with two attached vGPU devices) running both a high-performance 3D rendering application and a lightweight application can assign one vGPU device to each, eliminating resource contention and ensuring optimal performance for both tasks.
- **Expended Multi-Monitor Support:** A single vGPU device supports up to 4 monitors, which can be limiting for users requiring more displays. Multi-vGPU enables a VM to support additional monitors by attaching multiple vGPU devices, with each vGPU extending the monitor limit.
- **Engineering Simulations:** Run Computational Fluid Dynamics (CFD) simulations and other engineering workloads significantly faster, more smoothly, and securely when powered by multiple NVIDIA vGPUs with NVIDIA RTX Virtual Workstation (RTX vWS), versus CPU-only solutions. Performance improvements can reach up to 78% faster, depending on the specific workload and vGPU configuration.
- **Photorealistic Rendering:** Render scenes significantly faster with multiple vGPUs, allowing designers to iterate quickly and collaborate from almost anywhere. Performance improvements can reach up to 1.5x faster, depending on the rendering software, the complexity of the scenes, and the specific vGPU configuration.

For example, a VM configured with two NVIDIA L40S GPUs using multi-vGPU can run large-scale AI models more efficiently compared to a single GPU. Similarly, design firms performing intensive simulation can reduce processing time significantly by assigning multiple vGPUs to a single VM.

Applications scale differently with multiple vGPUs. Some are highly parallelizable and benefit significantly, while others may experience limited improvement. Ensure your application is designed to support multi-vGPU and leverages the appropriate libraries and APIs.

Multi-vGPU configurations allow for the assignment of up to 16 vGPUs with differing amounts of frame buffer to a single VM, provided all vGPUs belong to the same board type and series. For example, an L40S-24Q vGPU can be paired with an L40S-16Q vGPU, but cross-series combinations, such as pairing an L4-8Q vGPU with an A16-8Q vGPU, are not supported due to differences in GPU architecture generations.

Table 1: Multi-vGPU Compatibility

Hypervisor	vGPU Version	Supported GPUs, vGPUs, and Hypervisor Versions	Setup Instructions
------------	--------------	--	--------------------

Hypervisor	vGPU Version	Supported GPUs, vGPUs, and Hypervisor Versions	Setup Instructions
Citrix XenServer	All Active vGPU Releases	Citrix XenServer	Setting up Multi-vGPU on Citrix XenServer
Microsoft Azure Local	All Active vGPU Releases	Microsoft Azure Local	Setting up Multi-vGPU on Microsoft Azure Local
Microsoft Windows Server	Supported since 18.0	Microsoft Windows Server	Setting up Multi-vGPU on Microsoft Windows Server with Hyper-V
Red Hat Enterprise Linux with KVM	All Active vGPU Releases	Red Hat Enterprise Linux with KVM	Setting up Multi-vGPU on Linux KVM Reference Hypervisors VM using virsh / Setting up Multi-vGPU on Linux KVM Reference Hypervisors VM using the QEMU Command Line
Canonical Ubuntu with KVM	All Active vGPU Releases	Canonical Ubuntu with KVM	Setting up Multi-vGPU on Linux KVM Reference Hypervisors VM using virsh / Setting up Multi-vGPU on Linux KVM Reference Hypervisors VM using the QEMU Command Line
VMware vSphere	All Active vGPU Releases	VMware vSphere	Setting up Multi-vGPU on VMware vSphere

Note

The list of boards, vGPUs, and hypervisor versions applies to the latest vGPU release. For older versions and compatibility details, please visit the [NVIDIA Virtual GPU \(vGPU\) Software page](#).

vGPU Schedulers

vGPU Schedulers determine how GPU resources are shared across multiple virtual machines (VMs), balancing performance and ensuring fair access in multi-tenant environments. They allow administrators to customize resource allocation based on workload intensity and organizational priorities, ensuring optimal resource utilization and alignment with business needs. The scheduling mode determines how GPU time is divided among VMs and directly impacts factors like latency, throughput, and performance stability.

For workloads with varying demands, time slicing plays a critical role in determining scheduling efficiency. The vGPU scheduler time slice represents the duration a VM's work is allowed to run on the GPU before it is preempted. A shorter time slice reduces latency, making it ideal for latency-sensitive tasks like graphics applications. In contrast, longer time slices maximize throughput for compute-heavy workloads, such as CUDA applications, by minimizing context switching.

NVIDIA provides three scheduling modes— **Best Effort**, **Equal Share**, and **Fixed Share** — each designed to different workload requirements and environments.

Best Effort Scheduler

Best effort is the default scheduler for all supported GPU architectures, suitable for all environments with fluctuating workloads. The key principles of this scheduler are as follows:

- **Timesliced Round Robin Approach:** GPU resources are allocated to VMs in a timesliced manner. If a VM has no task or has used up its time slice, the scheduler will move to the next VM in the queue.
- **Non-Guaranteed Shares:** This scheduler does not guarantee a fixed or equal share of GPU cycles per VM. Instead, the allocation is determined by the current demand, which can lead to uneven distribution of GPU resources.
- **Optimized Utilization:** During idle periods, the scheduler maximizes resource utilization to enhance user density and Quality of Service (QoS).

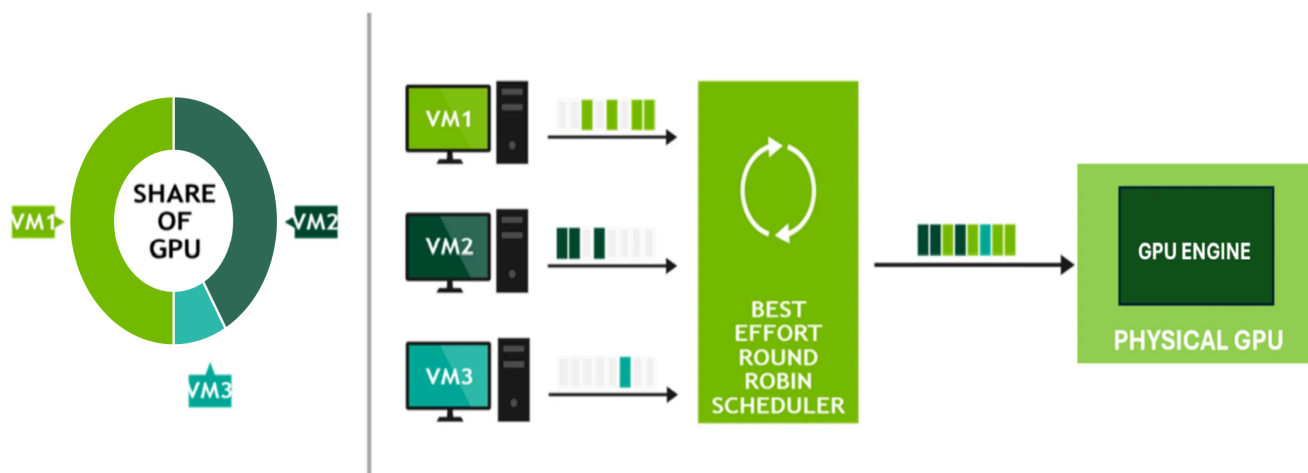


Figure 2: Best Effort Scheduler

Figure 2 is a simplified explanation of how the best effort scheduler works. It takes advantage of unused GPU time-slices from other VMs. Since workloads across virtual machines are not executed at the same time, and are not always GPU bound, the best effort scheduler allows a VM's share of the GPU's performance to exceed its expected share.

Equal Share Scheduler

This mode divides GPU resources equally among all active vGPUs, ensuring fair access to GPU cycles. GPU processing cycles are allocated with each vGPU receiving an equal share of GPU resources. Specifically, each vGPU gets $1/N$ of the available GPU cycles, where N is the number of vGPUs running on the GPU.

- **Deterministic Allocation:** Each vGPU is guaranteed a share of GPU cycles, regardless of demand, as long as the number of vGPUs does not change. If the number of vGPUs changes, the allocation will adjust accordingly to ensure fair distribution of resources across all vGPUs.
- **Idle Periods:** If a VM has no tasks during its allocated timeslice, the GPU resources assigned to it will remain idle rather than being reallocated to other VMs.
- **Dynamic Adjustment:** As vGPUs are added or removed, the scheduler dynamically reallocates resources to maintain equal distribution. This means a vGPU's performance can improve when other vGPUs stop or degrade when more vGPUs are added to the same GPU.
- **Fairness and Consistency:** Equal Share ensures fairness and consistency across workloads, making it ideal for environments prioritizing balanced resource allocation.

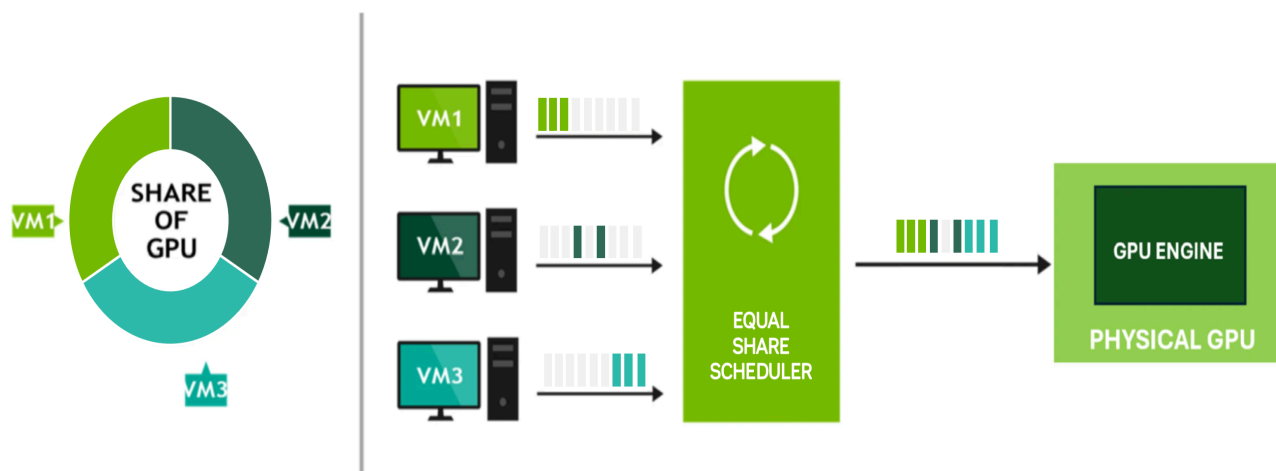


Figure 3: Equal Share Scheduler

Figure 3 is a simplified explanation of how the equal share scheduler works. Each VM is allocated an equal share of GPU. However, even if a VM does not fully utilize its assigned share (e.g., VM2 has an idle gap in its workload), that idle time remains part of the VM's schedule. The scheduler merges all workloads, including idle time, into a single execution list that is sent to the GPU engine, ensuring fair allocation of GPU power.

Fixed Share Scheduler

This mode allocates a dedicated, consistent portion of GPU resources to each VM, ensuring predictable availability and stable performance. Administrators can select a vGPU type, and the driver software determines the fixed allocation based on that choice. For example, a L4 GPU with four L4-6Q vGPUs allocates 25% of GPU cycles to each vGPU. As vGPUs are started or stopped, the share of the GPU's processing cycles allocated to each vGPU remains constant, ensuring that the performance of a vGPU remains unchanged. Fixed share is particularly useful in environments where performance consistency is critical, such as for high-priority applications. It also simplifies Proof of Concept (POC) testing by enabling reliable comparisons between physical and virtual workstations using benchmarks like SPECviewperf.

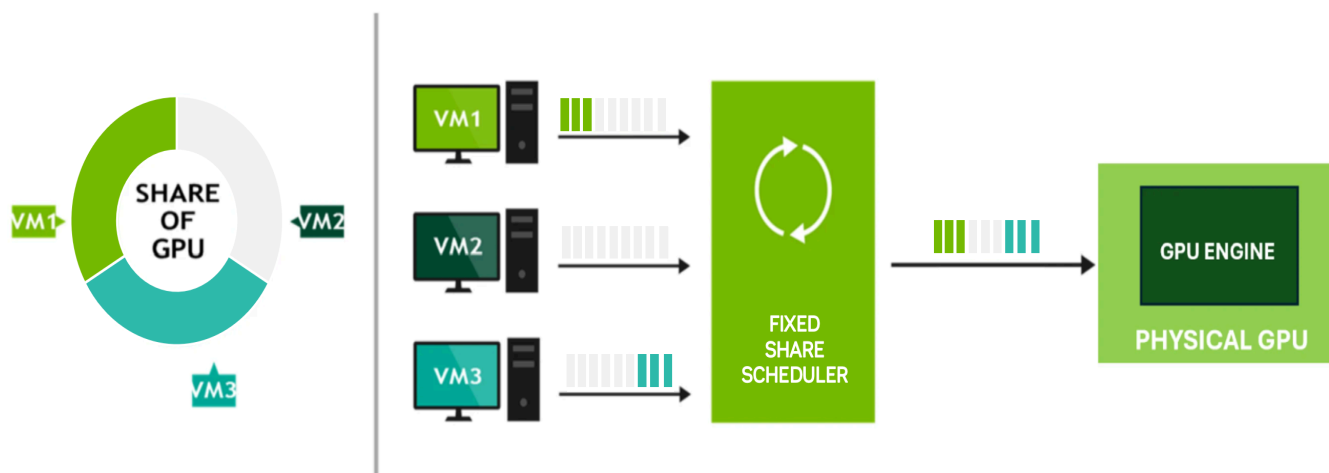


Figure 4: Fixed Share Scheduler

Figure 4 is a simplified example of how the fixed share scheduler works. In this scenario, the GPU supports a maximum of three vGPUs of the same type, but one VM (VM2) is powered off, leaving part of the GPU resources unused. VM1 and VM3 each receive a fixed 1/3rd share of GPU resources, while the remaining 1/3rd remains unallocated. These shares are fixed, meaning that the VMs will continue to get their predefined portions of GPU resources, regardless of the changing resource demands. This fixed allocation ensures that each VM has a guaranteed share, maintaining consistent performance even if workloads fluctuate.

The vGPU scheduler operates via a strict round-robin scheduling mechanism, enforcing fairness in GPU time allocation across VMs. Adjustments to the time slice are based on the scheduling frequency and an averaging factor. Scheduling frequency adjusts dynamically based on the number of active vGPUs:

- **Fewer than 8 vGPUs:** Default scheduling frequency is 480 Hz.
- **8 or more vGPUs:** Default scheduling frequency is 960 Hz.

To ensure optimal performance and resource utilization, administrators should configure scheduling modes based on workload requirements and GPU architecture capabilities. Shorter time slices are recommended for latency-sensitive workloads, such as graphics applications, while longer time slices are better suited for compute-intensive tasks to reduce context switching. Regularly monitor GPU performance using tools like `nvidia-smi` and adjust settings as necessary.

By default, Frame-Rate Limiter (FRL) is enabled to ensure balanced performance across multiple vGPUs on the same physical GPU, providing a smooth remote graphics experience. When using equal share or fixed share schedulers, FRL is automatically disabled to support consistent performance because these schedulers themselves maintain balanced

performance across VMs. For best effort schedulers, FRL can be manually disabled as detailed in the release notes for your hypervisor:

- [Disable or enable FRL on Canonical Ubuntu with KVM](#)
- [Disable or enable FRL on Citrix XenServer](#)
- [Disable or enable FRL on Linux KVM Reference Hypervisors](#)
- [Disable or enable FRL on Red Hat Enterprise Linux with KVM](#)
- [Disable or enable FRL on VMware vSphere](#)

The FRL setting is designed to give good interactive remote graphics experience but may reduce scores in benchmarks that depend on measuring frame rendering rates, as compared to the same benchmarks running on a pass-through GPU.

Starting with vGPU 20.0 and later releases, fixed-share scheduling is supported with heterogeneous vGPU configurations. The best-effort and equal-share schedulers were already supported.

For detailed guidance on configuring and adjusting scheduling policies to meet specific resource distribution needs, consult the NVIDIA vGPU User Guide:

- [Getting Information about the Scheduling Behavior of Time-Sliced vGPUs](#)
- [Tools for Changing Scheduling Behavior for Time-Sliced vGPUs](#)
- [Changing Scheduling Behavior for Time-Sliced vGPUs by Using the nvidia-smi Command](#)
- [Changing Scheduling Behavior for Time-Sliced vGPUs by Using the RmPVMRL Registry Key](#)

Live Migration

Live migration enables the seamless transfer of virtual machines (VMs) configured with NVIDIA vGPUs from one physical host to another without downtime. This capability allows organizations to maintain continuous operations during infrastructure changes, balancing workloads or reallocating resources with minimal disruption.

Live migration offers significant benefits across various operational scenarios:

- **Minimal Business Disruptions:** Perform hardware upgrades or repairs without impacting active workloads or user experiences.
- **Load Balancing:** Optimize resource allocation by shifting GPU-intensive VMs to less-utilized hosts.

- **Scalability and Agility:** Reallocate GPU resources or migrate VMs to hosts with better hardware to meet evolving operational needs.

Most virtualization platforms enable live migration for VMs with vGPUs, provided the source and destination hosts meet compatibility requirements. These platforms demonstrate how live migration can be seamlessly integrated into GPU-accelerated virtualized environments.

For live migration to be successful, certain prerequisites must be met:

- The destination host must have the same type of physical GPU as the source host.
- Both source and destination GPUs must have identical memory configurations (ECC enabled or disabled) and, for multi-vGPU configurations, identical NVlink topologies.
- Both the source and destination hosts must have compatible NVIDIA GPU drivers and hypervisor versions.

To enable seamless live migration of VMs with multiple vGPUs and minimize downtime or disruptions, the host hypervisors in a cluster must have the same board type and profiles.

During the migration process, the hypervisor replicates the source VM's system memory, CPU execution state, vGPU framebuffer, and vGPU execution state to the destination host in real time. Once replication is complete, the VM is switched to the new host, allowing applications to continue running without interruption and with minimal latency.

Live migration is not supported for VMs utilizing specific NVIDIA CUDA Toolkit features, including:

- [Unified memory](#)
- [Debuggers](#)
- [Profilers](#)

These features must be disabled for successful migration.

Table 2: Live Migration Compatibility

Hypervisor	vGPU Version	Guest OS Support	Setup Instructions
Canonical Ubuntu with KVM	vGPU 16.8, reintroduced in vGPU 17.2	Supported since Canonical Ubuntu 24.04 (Windows and Linux guests)	Migrating a VM Configured with vGPU on Linux KVM Reference Hypervisors

Hypervisor	vGPU Version	Guest OS Support	Setup Instructions
Citrix XenServer	All Active vGPU Releases	Supported since Citrix XenServer 8 and Citrix Hypervisor 8.2 (Windows only). Live migration on Citrix XenServer is facilitated through XenMotion .	Migrating a VM Configured with vGPU on Citrix XenServer
Microsoft Windows Server	Supported since vGPU 18.0	Supported since Windows Server 2025	Migrating a VM Configured with vGPU on Microsoft Windows Server
Red Hat Enterprise Linux with KVM	vGPU 16.6, reintroduced in vGPU 17.2	Supported since RHEL 9.4 (Windows and Linux guests)	Migrating a VM Configured with vGPU on Linux KVM Reference Hypervisors
VMware vSphere	All Active vGPU Releases	Supported on all supported releases of VMware vSphere (Windows and Linux guests). Live migration on VMware vSphere is facilitated through VMware vMotion .	Migrating a VM Configured with vGPU on VMware vSphere

Note

The guest OS support applies to the latest vGPU release. For older versions and compatibility details, please visit the [NVIDIA Virtual GPU \(vGPU\) Software page](#).

Heterogeneous vGPU

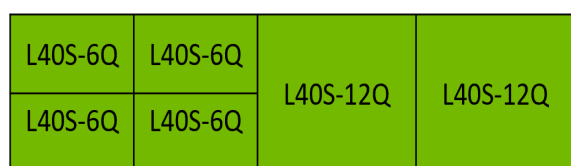
Heterogeneous vGPU allows a single physical GPU to support multiple vGPU profiles with different memory allocations (frame buffer sizes) simultaneously. This configuration is particularly beneficial for environments where virtual machines (VMs) have diverse GPU

requirements. By enabling the same physical GPU to host vGPUs with varying profile sizes, heterogeneous vGPU allows VMs to access only the GPU resources they need based on their specific workload, optimizing overall resource usage and preventing underutilization.

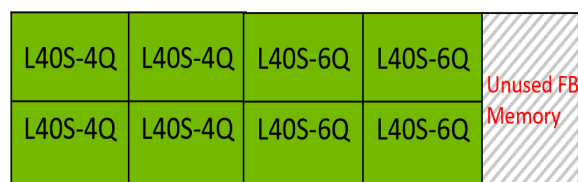
In heterogeneous vGPU, a GPU can support different types of vGPU profiles, such as A, B, and Q series, each with unique performance characteristics and memory allocations. For instance, an NVIDIA L4 GPU can host an L4-8Q (8 GB frame buffer) alongside an L4-2B (2 GB frame buffer), allowing high-demand applications like 3D rendering to coexist with lighter workloads, such as office productivity tasks. This flexibility helps data centers and cloud providers meet diverse user demands efficiently, tailoring vGPU configurations to the specific needs of each VM without requiring multiple GPUs. For example, the following combinations of vGPUs can reside on the same NVIDIA L4 GPU simultaneously:

- L4-2B and L4-2Q
- L4-2Q and L4-4Q
- L4-2B and L4-4Q

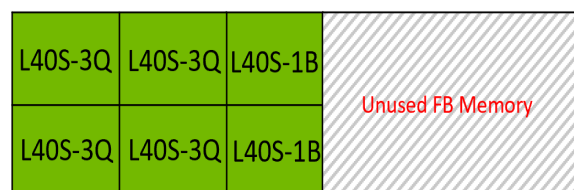
4 x L40S-6Q and 2 x L40S-12Q on GPU1:



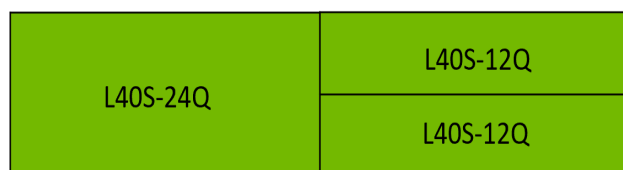
4 x L40S-4Q and 4 x L40S-6Q on GPU1:



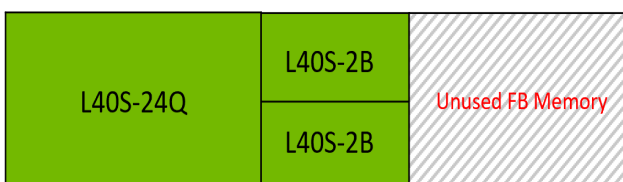
4 x L40S-3Q and 2 x L40S-1B on GPU1:



1 x L40S-24Q and 2 x L40S-12Q on GPU2:



1 x L40S-24Q and 2 x L40S-2B on GPU2:



1 x L40S-8Q, 2 x L40S-12Q and 1 x L40S-16Q on GPU2:

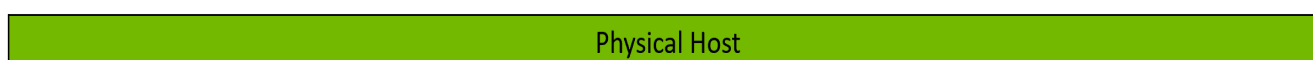
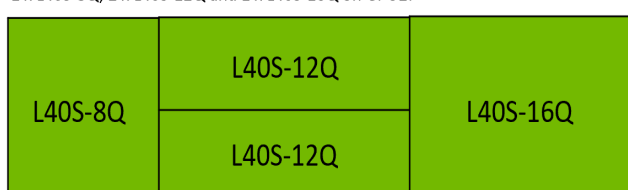


Figure 5: Example Heterogeneous vGPU Configurations for NVIDIA L40S

Heterogeneous vGPU is particularly advantageous for environments with varied workloads, such as:

- **Multi-tenant environments:** Allows each tenant to receive GPU resources tailored to their application needs, maximizing cost-efficiency.
- **Virtual Desktop infrastructure (VDI):** Enables a range of applications, from graphics-intensive workstations to standard productivity tasks, to operate on the same GPU.
- **Cloud environments:** Cloud Service Providers (CSPs) can flexibly deploy vGPUs to meet various customer requirements within a single infrastructure.

Using heterogeneous vGPU with vGPUs requires careful planning, monitoring, and adjustments to achieve the best results. A recommendation is to use distributed load balancers, such as VMware DRS or Nutanix AHV Scheduling, that can optimize workload distribution across a cluster, ensuring that hosts are evenly utilized despite the varying demands of different vGPU profiles. Additionally, vGPU Schedulers manage how resources are allocated within a host, prioritizing specific VMs and maintaining balanced performance in heterogeneous vGPU environments.

In cases where a distributed load balancer is unavailable, understanding the specific needs of your applications becomes essential. For instance, applications like CAD or video editing often require more frame buffer memory, while deep learning and AI workloads rely on greater compute power. Categorizing VMs based on their vGPU requirements, such as high, medium, or low, can assist in planning resource allocation within a cluster. However, achieving an effective balance still requires careful monitoring and manual adjustments to address changes in workload demands. To maintain balanced workload distribution and optimize GPU performance, it is recommended to avoid placing multiple VMs with large vGPU profiles on a single GPU. Instead, strive to balance the GPU's VMs between compute-intensive and memory-intensive workloads. This approach helps prevent resource contention and ensures efficient utilization of GPU resources.

In heterogeneous vGPU, the maximum number of vGPU instances of a given size may be lower than the total framebuffer size would allow, due to packing and alignment constraints of other vGPU resources. For example, an L40S GPU with 48 GB of GPU memory can support:

- 6 instances of the L40S-8Q profile in equal-size mode.
- 4 instances of the L40S-8Q profile in heterogeneous vGPU.

Table 3: L40S-8Q vGPU Profile

Virtual GPU Type	Frame Buffer (MB)	Maximum vGPUs per GPU in Equal-Size Mode	Maximum vGPUs per GPU in Heterogeneous vGPU
L40S-8Q	8192	6	4

When a GPU is configured for heterogeneous vGPU, its behavior varies depending on the hypervisor during events such as a host reboot, NVIDIA Virtual GPU Manager reload, or GPU reset. On Linux KVM Reference Hypervisors, the GPU reverts to its default mode, whereas on VMware vSphere, the GPU remains in heterogeneous vGPU. Starting with vGPU 20.0 and later releases, the fixed share scheduler is also supported in heterogeneous vGPU configurations. The best effort and equal share schedulers were already supported. The vGPU placements that a GPU with heterogeneous vGPU supports depend on the total amount of frame buffer that the GPU has. For detailed information on vGPU placements for a specific GPU, refer to [vGPU Placements for GPUs in Heterogeneous vGPU](#).

Heterogeneous vGPU is supported on Volta and later GPUs.

Table 4: Heterogeneous vGPU Compatibility

Hypervisor	vGPU Version	Setup Instructions
Canonical Ubuntu with KVM	Supported since vGPU 17.0	Configuring a GPU for Heterogeneous vGPU on Linux KVM Reference Hypervisors / Placing a vGPU on a Physical GPU with Heterogeneous vGPU on Linux KVM Reference Hypervisors
Red Hat Enterprise Linux with KVM	Supported since vGPU 17.0	Configuring a GPU for Heterogeneous vGPU on Linux KVM Reference Hypervisors / Placing a vGPU on a Physical GPU with Heterogeneous vGPU on Linux KVM Reference Hypervisors
VMware vSphere	Supported since vGPU 17.2	Configuring a GPU for Heterogeneous vGPU on VMware vSphere Using vCenter Server / Configuring a GPU for Heterogeneous vGPU on VMware vSphere Using the esxcli Command

Example Configurations for Heterogeneous vGPU:

Table 5: Heterogeneous vGPU Example Configurations

vGPU Profiles	Example Configuration	Use Case
2x L40S-8Q, 1x L40S-2B	2 VMs with L40S-8Q profiles for high-end graphics tasks, 1 VM with L40S-2B for lighter workloads	3D Rendering + Basic Applications
1x L4-8Q, 2x L4-2B	1 VM with L4-8Q profile for CAD work, 2 VMs with L4-2B for general office tasks	CAD + Basic Applications

[Table 5](#) highlights two sample configurations showcasing how heterogeneous vGPU allows different vGPU profiles to coexist on a single GPU, serving varied workloads simultaneously. By supporting a range of memory allocations, heterogeneous vGPU maximizes GPU efficiency and meets specific application needs, from intensive graphics to productivity tasks, within the same infrastructure.

Suspend-Resume

The suspend-resume feature allows virtual machines (VMs) configured with NVIDIA vGPUs to be temporarily paused (suspended) and later resumed without losing their state. During suspension, the entire VM state is saved to disk. Upon resumption, this state is restored, enabling workloads to continue seamlessly. This feature provides operational flexibility and resource optimization for GPU-accelerated virtualized environments. When a VM is suspended, GPU and compute resources are freed up, allowing administrators to prioritize high-demand tasks or accommodate additional workloads.

Suspend-resume is particularly useful for planned host maintenance, resource balancing, and pausing non-critical workloads during off-peak hours. It also serves as a fallback option in scenarios where [Live Migration](#), which transfers a VM’s state in real time with minimal or no downtime, is not feasible.

This feature is valuable for downtime management, allowing VMs to be suspended during host maintenance and resumed later without loss of state. It also supports workload redistribution by temporarily pausing VMs to shift resources between hosts. For

development and testing, suspend-resume ensures that environments can be paused and resumed while maintaining their exact state, saving both time and resources.

To ensure compatibility and prevent failures during suspend-resume operations:

- **For Suspend-Resume on the Same Host:** No additional compatibility checks are required as the host's hardware and software configurations remain unchanged.
- **For Suspend-Resume Across Hosts:**
 - The destination host must have the same type of physical GPU and a compatible vGPU manager version.
 - The VM must be running and configured with a licensed vGPU.
 - Both hosts must have matching memory configurations (ECC enabled or disabled) and GPU topologies, including NVLink widths.

For more details on compatibility requirements for the NVIDIA vGPU Manager and guest VM driver, refer to the following:

- [Canonical Ubuntu with KVM](#)
- [Citrix XenServer](#)
- [Red Hat Enterprise Linux with KVM](#)
- [VMware vSphere](#)

Note

While live migration generally allows resuming a suspended VM on any compatible vGPU host manager, a current bug in Linux/KVM distributions limits suspend/resume/migration to hosts with an identical vGPU manager version. Fixes for this limitation are expected this year.

To optimize the use of the suspend-resume feature, administrators should plan suspensions during periods of low workload activity to minimize disruptions. Compatibility between hosts should be thoroughly checked, particularly for memory configurations and GPU topology, when operating across multiple hosts. Regularly monitoring GPU resource utilization and planning suspension of non-critical workloads can free up resources for high-priority tasks. For development and testing workflows, suspend-resume offers a practical solution for preserving VM states, saving time and resources during iterative processes.

When a VM is suspended, its memory state is written to disk. Therefore, it is important to ensure the VM is allocated an appropriate amount of memory. Excessive memory allocation can significantly increase suspend and resume times. To ensure the best possible user

experience, use high-performance disks, like solid-state drives (SSDs), to achieve faster suspend and resume operations.

Unlike live migration, suspend-resume involves downtime during suspension and resumption, making it less suitable for workloads requiring real-time continuity. Additionally, operations across different hosts require strict compatibility in GPU type, vGPU manager version, memory configuration, and NVLink topology.

Suspend-resume is supported on all GPUs that support vGPU, but compatibility varies by hypervisor, release version, and guest operating system.

Table 6: Suspend-Resume Compatibility

Hypervisor	vGPU Version	Guest OS Support	Setup Instructions
Canonical Ubuntu with KVM	Supported since vGPU 17.2	Supported for both Windows and Linux guests.	Suspending and Resuming a VM Configured with vGPU on Linux KVM Reference Hypervisors
Citrix XenServer	All Active vGPU Releases	Supported since Citrix XenServer 8 and Citrix Hypervisor 8.2 (Windows only).	Right-click the VM and select suspend or resume from the menu.
Red Hat Enterprise Linux with KVM	Supported since vGPU 17.2	Supported for both Windows and Linux guests.	Suspending and Resuming a VM Configured with vGPU on Linux KVM Reference Hypervisors
VMware vSphere	All Active vGPU Releases	Supported on all supported releases of VMware vSphere (Windows and Linux guests).	Suspending and Resuming a VM Configured with vGPU on VMware vSphere

Note

The guest OS support applies to the latest vGPU release. For older versions and compatibility details, please visit the [NVIDIA Virtual GPU \(vGPU\) Software page](#).

Unified Virtual Memory

Unified virtual memory provides a single memory address space that is accessible by both the CPUs and GPUs in a system. It creates a managed memory pool that allows data to be allocated and accessed by code running on any CPU or GPU, thus simplifying programming and enhancing performance for GPU-accelerated workloads. This feature eliminates the need for apps to program data transfers between CPU and GPU memory, enabling seamless resource utilization.

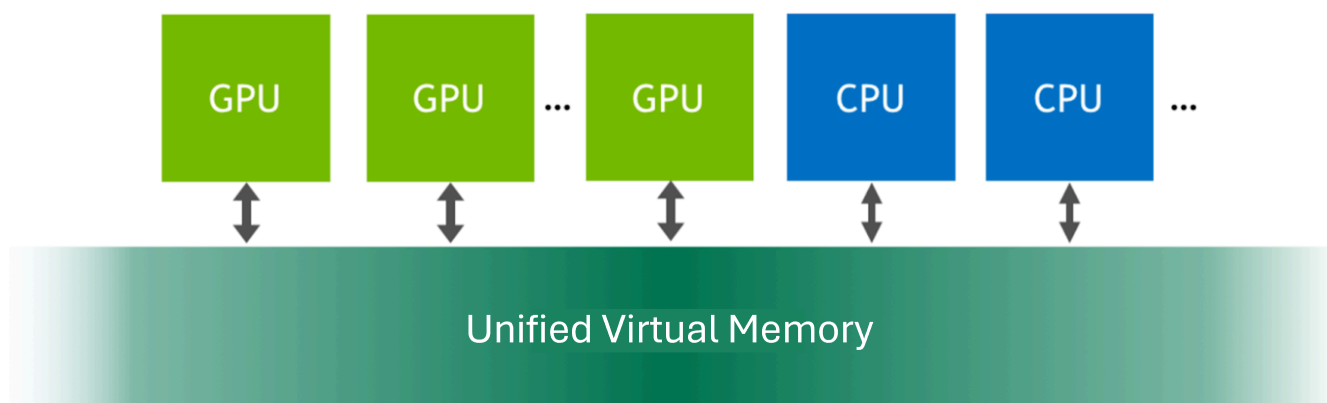


Figure 6: Unified Virtual Memory

Some applications require unified virtual memory for efficient operation or may not run at all without it. To ensure compatibility, users need to enable it on their vGPU when running these applications. Common use cases include:

- **GPU-Accelerated Applications:** Ideal for workloads requiring frequent CPU-GPU data exchange, such as machine learning, data analytics, scientific simulations, real-time simulations, and data-intensive workflows.
- **Linux-Based Applications:** Supports resource-intensive workloads running on Linux operating systems.
- **CUDA Development:** Facilitates profiling and optimization of GPU-accelerated applications using the NVIDIA CUDA Toolkit.

To ensure optimal performance and stability with unified virtual memory configurations, always follow hypervisor-specific guidelines and use a licensed Q-series vGPU profile, or C-

series vGPU profiles that are packaged with NVIDIA AI Enterprise, with sufficient resources to meet workload demands. NVIDIA CUDA Toolkit profilers can be enabled on virtual machines as needed to monitor memory usage, analyze access patterns, identify potential bottlenecks, and provide tools for application profiling and optimization. However, they should only be used during debugging or performance tuning. They should be enabled when trying to debug or tune app performance.

You can enable the following CUDA Toolkit profiler features for a vGPU VM:

- NVIDIA Nsight™ Compute
- NVIDIA Nsight Systems
- CUDA Profiling Tools Interface (CUPTI)

These profilers help identify performance issues and optimize GPU workloads by analyzing kernel performance and memory access patterns. For step-by-step instructions, see [Enabling CUDA Toolkit Profilers for a vGPU VM](#).

When profiling with Nsight Compute or CUPTI, clocks are automatically locked for multipass profiling when profiling starts and unlocked when it ends. For periodic sampling use cases like Nsight Systems profiling, clocks are not locked. Administrators should monitor the impact of clock locking on workload performance, particularly in shared environments.

Regularly check kernel versions, GPU drivers, and hardware configurations to maintain compatibility and avoid disruptions. Administrators should also utilize NVIDIA driver logs and hypervisor monitoring tools to track GPU resource allocation and identify issues early, preventing resource contention or performance degradation in multi-VM environments.

Unified virtual memory is supported **only** on Linux operating systems, and it is limited to Q-series vGPUs or C-series vGPU profiles that are packaged with NVIDIA AI Enterprise, that allocate the entire frame buffer of a unified virtual memory-compatible physical GPU. Fractional time-sliced vGPUs are not compatible with unified virtual memory.

Enabling unified virtual memory disables vGPU migration for the VM, which could limit operational flexibility in environments requiring live migration. Additionally, NVIDIA CUDA Toolkit profilers can only be used on one VM at a time per GPU. Profiling multiple CUDA contexts simultaneously is not supported, and data is collected separately for each context. A VM with CUDA profilers enabled cannot be migrated. These limitations must be considered when planning profiling in environments with multiple vGPU VMs sharing the same GPU.

Table 7: Unified Virtual Memory Compatibility

Hypervisor	vGPU Version	Supported GPUs, vGPUs, and Hypervisor Versions	Setup Instructions
Canonical Ubuntu with KVM	All Active vGPU Releases	Canonical Ubuntu with KVM	Enabling Unified Virtual Memory for a vGPU on Linux KVM Reference Hypervisors
Citrix XenServer	All Active vGPU Releases	Citrix Xenserver	Enabling Unified Virtual Memory for a vGPU on Citrix XenServer
Microsoft Azure Local	All Active vGPU Releases	Microsoft Azure Local	Enabled by default
Microsoft Windows Server	Supported since vGPU 18.0	Microsoft Windows Server	Enabled by default
Red Hat Enterprise Linux with KVM	All Active vGPU Releases	Red Hat Enterprise Linux with KVM	Enabling Unified Virtual Memory for a vGPU on Red Hat Enterprise Linux KVM
VMware vSphere	All Active vGPU Releases	VMware vSphere	Enabling Unified Virtual Memory for a vGPU on VMware vSphere

Note

The list of boards and vGPUs applies to the latest vGPU release. For older versions and compatibility details, please visit the [NVIDIA Virtual GPU \(vGPU\) Software page](#).

Unified virtual memory is disabled by default and must be explicitly enabled for each vGPU that requires it. This is done by setting a vGPU plugin parameter for the VM.

Note

Unified virtual memory is enabled by default for Microsoft Azure Local and Microsoft Windows Server. If you do not want to use unified virtual memory, you must disable it individually for each vGPU by setting a vGPU plugin parameter.

Deep Learning Super Sampling (DLSS)

NVIDIA Deep Learning Super Sampling (DLSS) is an AI-driven upscaling technology that leverages deep learning models and NVIDIA Tensor Cores to reconstruct high-resolution frames from lower-resolution inputs. DLSS significantly boosts graphical performance while maintaining image quality, allowing applications to render at lower resolutions and upscale in real-time with minimal overhead.

DLSS dramatically improves performance by using AI to analyze motion data and sequential frames, generating additional high-quality frames for smoother visuals. It continuously improves over time, with AI models trained on NVIDIA supercomputers to deliver better image quality and performance across more games and applications.

DLSS provides specific features tailored to various use cases:

- **DLSS Super Resolution:** Boosts performance for all GPUs by using AI to output higher resolution frames from a lower resolution input. DLSS samples multiple lower resolution images and uses motion data and feedback from prior frames to reconstruct native quality images.
- **Deep Learning Anti-aliasing (DLAA):** Provides higher image quality for all GPUs with an AI-based anti-aliasing technique. DLAA uses the same Super Resolution technology developed for DLSS, reconstructing a native resolution image to maximize image quality.

DLSS is primarily used in graphically intensive applications such as games, 3D rendering, CAD and virtualized workstations, where performance and visual fidelity are critical. **It is supported in virtualized environments through NVIDIA vGPU software using NVIDIA RTX Virtual Workstations (vWS).** For a comprehensive and up-to-date list of the over 400 games and applications utilizing DLSS, visit the [NVIDIA DLSS and RTX Supported Titles Page](#).

DLSS is beneficial in a variety of GPU-accelerated scenarios, including:

- **Cloud Gaming:** Services use DLSS to stream popular AAA titles like Fortnite, Cyberpunk 2077 and Microsoft Flight Simulator at high frame rates while conserving GPU resources, delivering smooth and immersive experiences for end-users.
- **3D Design and Visualization:** Tools like Unity HDRP and Autodesk VRED use DLSS to accelerate real-time rendering workflows, enhancing architectural visualization and product design.
- **Virtual Workstations:** Engineers and creators in virtualized environments use DLSS with NVIDIA RTX Virtual Workstations (vWS) to reduce rendering times while maintaining precision in simulations and graphics.

To ensure optimal performance and seamless integration of DLSS in virtualized environments, administrators must use supported GPUs and RTX Virtual Workstation (vWS). During setup, it is crucial to ensure that the NVIDIA graphics driver detects a compatible GPU, as DLSS components will only install when a supported GPU is present. Additionally, always verify that your application supports DLSS and is compatible with the required APIs, such as DirectX 11, DirectX 12, or Vulkan. Developers should download and install the appropriate DLSS plugin for their game engine to streamline the implementation process and enable DLSS features effectively.

For DLSS Super Resolution, start with the Quality mode for the best image fidelity. If higher performance is required, switch to Balanced or Performance modes. Super Resolution performs best at higher resolutions like 1440p or 4K, while lower resolutions such as 1080p may diminish benefits and image quality.

Finally, for DLAA (Deep Learning Anti-Aliasing), enable this feature if your system already achieves high native frame rates without DLSS Super Resolution. DLAA can improve image quality significantly compared to traditional anti-aliasing techniques like Temporal Anti-Aliasing (TAA) and works best in visually demanding games like Diablo IV.

For Canonical Ubuntu with KVM, Citrix XenServer, Microsoft Windows Server, Red Hat KVM and VMware vSphere, the following GPUs support DLSS:

- NVIDIA L2
- NVIDIA L20
- NVIDIA L20 liquid cooled
- NVIDIA L4
- NVIDIA L40
- NVIDIA L40S
- NVIDIA RTX 5000 Ada

- NVIDIA RTX 5880 Ada
- NVIDIA RTX 6000 Ada
- NVIDIA A2
- NVIDIA A10
- NVIDIA A16
- NVIDIA A40
- NVIDIA RTX A5000
- NVIDIA RTX A5500
- NVIDIA RTX A6000
- Tesla T4
- Quadro RTX 6000
- Quadro RTX 6000 passive
- Quadro RTX 8000
- Quadro RTX 8000 passive

Supported DLSS versions: 2.0. Version 1.0 is not supported.

For Microsoft Azure Local, the following GPUs support DLSS:

- NVIDIA L2
- NVIDIA L20
- NVIDIA L20 liquid cooled
- NVIDIA L4
- NVIDIA L40
- NVIDIA L40S
- NVIDIA RTX 5000 Ada
- NVIDIA RTX 5880 Ada
- NVIDIA RTX 6000 Ada
- NVIDIA A2
- NVIDIA A10
- NVIDIA A16
- NVIDIA A40

Supported DLSS versions: 2.0. Version 1.0 is not supported.

Note

This list of GPUs applies to the latest vGPU release. For older versions and compatibility details, please visit the [NVIDIA Virtual GPU \(vGPU\) Software page](#).

Table 8: DLSS Compatibility

Hypervisor	vGPU Version
Canonical Ubuntu with KVM	All Active vGPU Releases
Citrix XenServer	All Active vGPU Releases
Microsoft Azure Local	All Active vGPU Releases
Microsoft Windows Server	Supported since vGPU 18.0
Red Hat Enterprise Linux with KVM	All Active vGPU Releases
VMware vSphere	All Active vGPU Releases

For access to DLSS SDKs, plugins, system requirements, and detailed integration guides, visit the [NVIDIA DLSS Developer Portal](#). NVIDIA graphics driver components required for DLSS are installed only if a supported GPU is detected during the driver installation process. To ensure proper configuration in vGPU environments, if VM templates are being created with the driver pre-installed, the template must be created from a VM that is configured with a supported GPU at the time of the driver installation. Additionally, only applications that use `nvngx_dlss.dll` version 2.0.18 or newer are supported.

Device Groups

Device Groups provide an abstraction layer for multi-device virtual hardware provisioning. They enable platforms to assign an optimal set of physically connected devices (e.g., GPUs connected via NVLink or a GPU-NIC pair) as a single logical unit by virtual machines. This abstraction ensures the highest possible performance for workloads that rely on low-latency and high-bandwidth communication between components.

Device groups can be composed of two or more hardware devices that share a common PCIe switch or devices that share a direct interconnect between each other. These groups are automatically detected at the hardware level and surfaces as a single selectable unit.

This simplifies virtual hardware assignment, making it easier to configure systems for specific low-latency and high-bandwidth workloads.

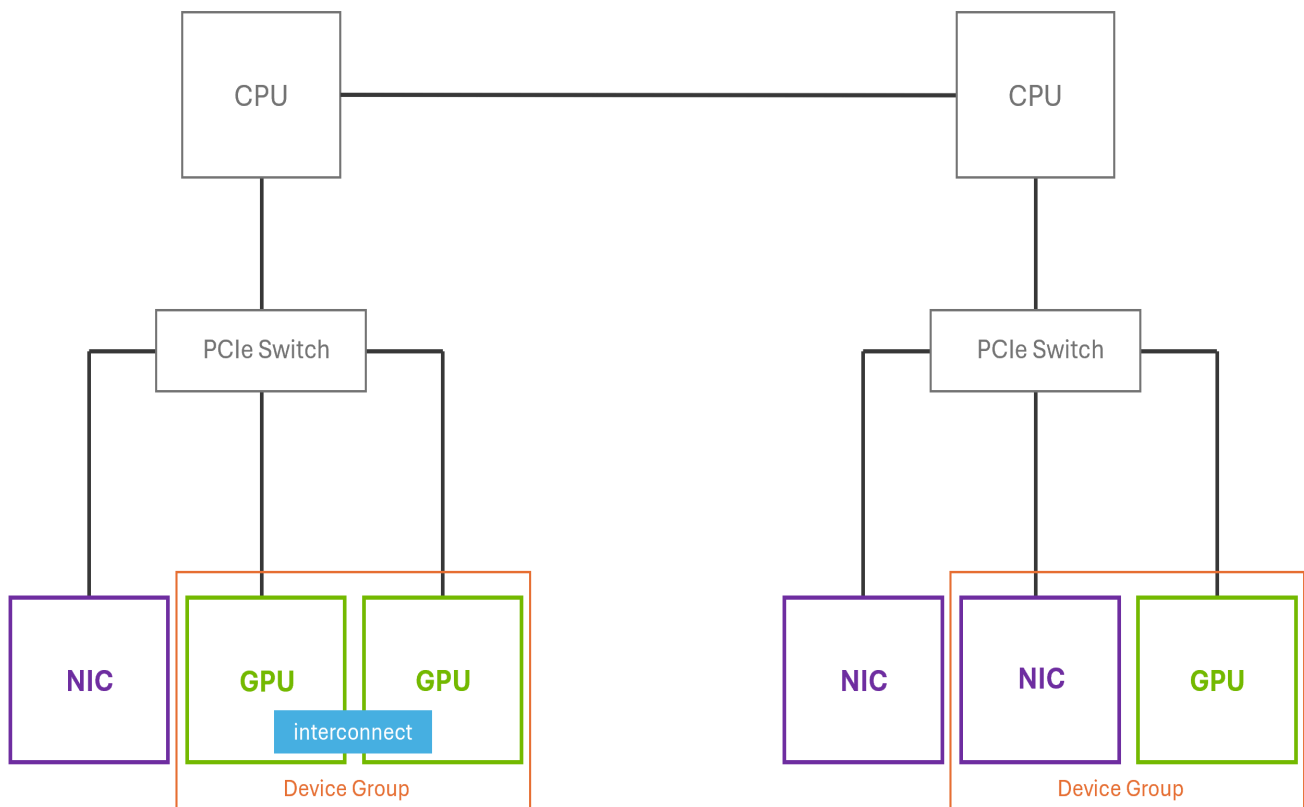


Figure 7: Device Groups

Figure 7 illustrates how devices that share a common PCIe switch or a direct GPU interconnect can be automatically discovered and grouped as a device group. On the right side, we can see that although two NICs are connected to the same PCIe switch as the GPU, only one NIC is included in the device group. This is because the NVIDIA driver identifies and exposes only the GPU-NIC pairings that meet the necessary criteria for optimized communication paths. Adjacent NICs that do not satisfy these requirements are excluded.

Device groups make hardware consumption easier and more reliable, particularly for AI/ML, high-performance computing, and data-intensive workloads that benefit from close coordination between GPUs or between GPUs and NICs. They are useful in cases where:

- **Multi-GPU or GPU-NIC configurations require direct hardware connection:** Device groups ensure that the full set of directly connected components (e.g., NVLink-connected GPUs or GPU-NIC pairs) are assigned together to the VM, enabling low-latency, high-bandwidth communication for performance sensitive workloads.
- **Topology consistency must be preserved:** Unlike manual device assignment, device groups guarantee correct placement across PCIe switches and interconnects, even after reboots or events like live migration.

- **Operational simplicity and reliability are critical:** Device groups eliminate the need for scripting or manual tracking of PCIe topology, reducing the risk of misconfiguration and enabling smarter scheduling through native platform awareness.

Device Groups on VMware vSphere

Device groups were introduced in VMware vSphere 8. They are tightly integrated with VMware vSphere Distributed Resource Scheduler (DRS), allowing VMs to be placed on hosts that meet specific hardware constraints. When a user adds a device group to a VM, it informs DRS that the VM must be placed on a host where the grouped devices exist together. This improves visibility and predictability in VM placement and ensures that performance-critical workloads get the topology they require. Device group assignments persist across host reboots, eliminating the need for reconfiguration or scripting after restart.

Device groups in VMware vSphere are limited to full physical GPU devices and full-profile time-sliced vGPUs. Multi-Instance GPU (MIG) profiles and MIG-backed vGPUs are not supported. Only VMware vSphere 8 and newer support device groups. They are currently restricted to GPUs and NICs and require specific PCIe topology.

To assign a device group to a VM, Navigate to the VM's **Edit Settings** panel in the VMware vSphere Client. Under **Add New Device** → **Other Devices** → **PCIe Device**, you'll see a list of available individual devices and device groups on the hosts in the cluster. Select the appropriate device group and save the configuration.

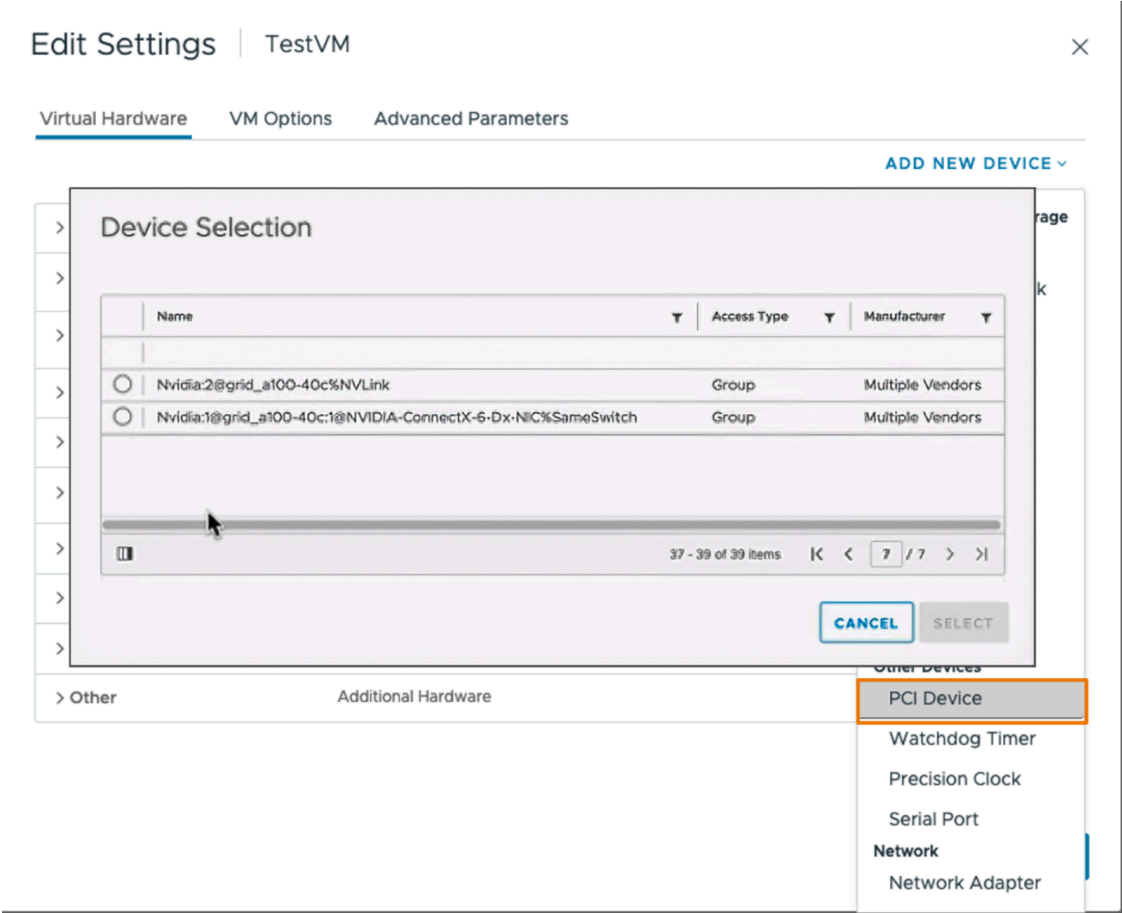


Figure 8: Selecting a Device Group

Table 9: Device Groups Compatibility

Hypervisor	vGPU Version	Supported GPUs, vGPUs, and Hypervisor Versions
VMware vSphere	Supported since vGPU 15.0	<u>All GPUs, vGPUs, and Hypervisor Versions supported in VMWare vSphere 8 and later</u>

Multi-Instance GPU (MIG)

Multi-Instance GPU (MIG) enables spatial partitioning of a single physical GPU chip into multiple isolated instances. Each instance functions as a dedicated mini-GPU with its own Streaming Multiprocessors (SMs) and memory subsystem. These instances operate independently, ensuring predictable performance and eliminating resource contention.

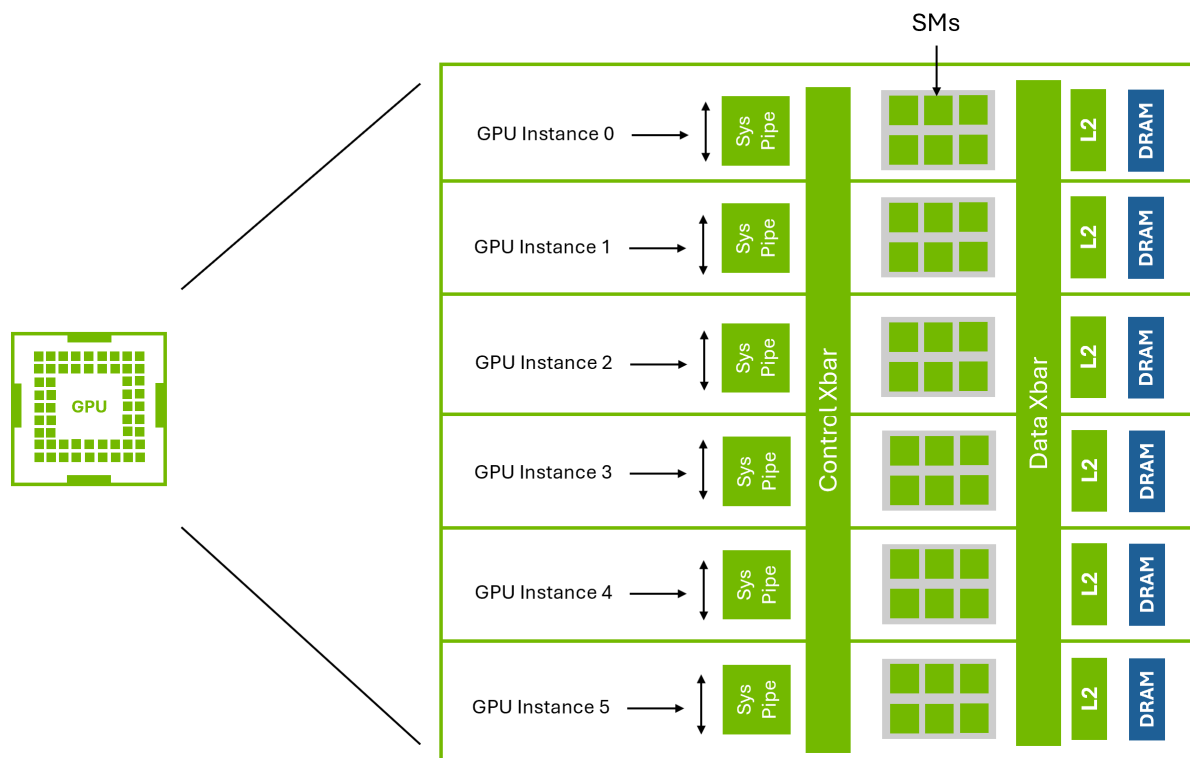


Figure 9: Multi-Instance GPU (MIG)

Figure 9 illustrates NVIDIA's MIG technology, which partitions a single GPU into multiple isolated instances at the hardware level.

NVIDIA Virtual GPU (vGPU) enables multiple virtual machines (VMs) to have simultaneous, direct access to a single physical GPU. This allows VMs to benefit from high graphics and compute performance, along with full application compatibility—while delivering the cost-effectiveness and scalability of sharing GPU resources across workloads.

Traditionally, vGPU has facilitated this with temporal partitioning, where a scheduler allocates each VM exclusive access to the GPU pipeline in turn through time slicing. This method is known as **Time-Sliced vGPU**. However, in high-density scenarios where the majority of VMs are running resource intensive workloads, scheduling overheads could outweigh time-slices, impacting the overall performance.

In addition to traditional time-sliced vGPU, the introduction of MIG presents another mode of virtualization with MIG-Backed vGPU, enabled for Graphics as well as Compute workloads with the RTX PRO 6000 Blackwell Server Edition and the RTX PRO 4500 Blackwell Server Edition. When MIG and vGPU are combined, they enable powerful deployment options. The following sections highlight the key capabilities unlocked by this integration.

MIG-Backed vGPU

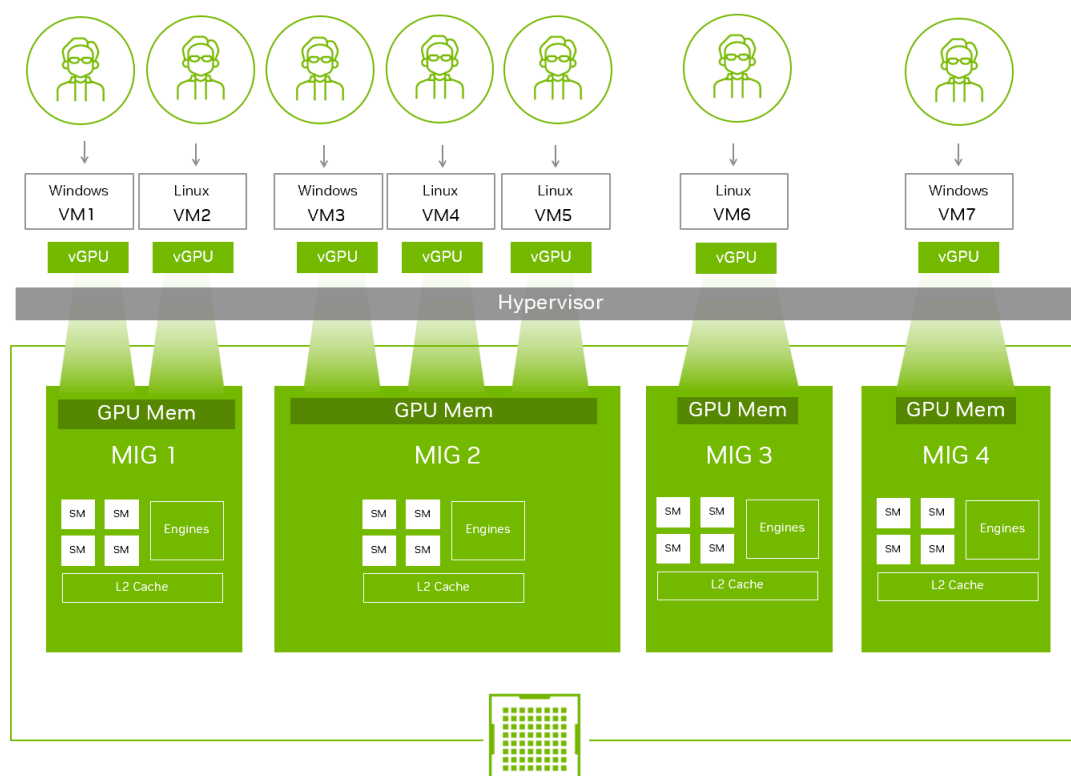
MIG-backed 1:1 vGPU assigns one vGPU to one MIG slice, providing full hardware isolation per VM.

MIG-backed time-sliced vGPU allows vGPUs to be created from individual MIG slices and assigned to virtual machines. This model combines MIG's hardware-level spatial partitioning with the temporal partitioning capabilities of vGPU, offering flexibility in how GPU resources are shared across workloads.

The NVIDIA RTX PRO Blackwell Server Edition GPUs enable MIG-backed vGPU, with each GPU model offering different density capabilities.

- On the NVIDIA RTX PRO 6000 Blackwell Server Edition, **up to 4 MIG slices** can be created on a single GPU. Within each MIG slice, **1 to 12 vGPUs** can be created, enabling **up to 48 virtual machines** to share a single physical GPU.
- On the NVIDIA RTX PRO 4500 Blackwell Server Edition, **up to 2 MIG slices** can be created on a single GPU. Within each MIG slice, **1 to 8 vGPUs** can be created, enabling **up to 16 virtual machines** to share a single physical GPU.

In both cases, each vGPU instance can be assigned to a separate VM while still benefiting from the isolation boundaries provided by MIG.



RTX PRO 6000 Blackwell Server Edition

Figure 10: MIG-Backed Time-Sliced vGPU

[Figure 10](#) shows how each MIG slice on the NVIDIA RTX PRO 6000 Blackwell can be time-sliced across multiple VMs—supporting up to 12 per slice—to maximize user density while maintaining performance isolation through hardware-level partitioning.

Universal MIG: Graphics and Compute Enablement

Thanks to Universal MIG, the NVIDIA RTX PRO 6000 Blackwell is the first data center GPU to support both compute and graphics workloads across MIG instances. This marks a significant evolution, as previous MIG-capable GPUs were limited to compute-only partitions. Universal MIG is also supported on the NVIDIA RTX PRO 4500 Blackwell Server Edition.

By enabling graphics acceleration within MIG slices, Universal MIG expands the range of supported workloads, making it suitable for mixed-use environments that require both AI/ML compute and interactive visualization or rendering. This added flexibility allows organizations to consolidate diverse workloads on a single GPU while still benefiting from MIG's strict resource isolation and predictable performance.

Note

Only RTX PRO 6000 Blackwell Server Edition and the RTX PRO 4500 Blackwell Server Edition supports Universal MIG with vGPU technology. RTX PRO 6000 Blackwell Workstation Edition and Max-Q Edition do NOT support vGPU technology.

vGPU license enforcement applies to MIG-Backed vGPU. For more information, refer to the [NVIDIA License System documentation](#).

Secure Multitenancy

MIG alone does not support multi-tenancy. To achieve true multi-tenancy, vGPU is required. When combined with MIG, vGPU allows the hypervisor to assign individual MIG-backed vGPUs to individual VMs. While both time-sliced vGPU and MIG-backed vGPU enable multi-tenancy with GPU resource sharing, MIG-backed vGPU provides hardware-level isolation and significantly improves quality of service (QoS). vGPUs within a MIG slice still share resources temporally, but isolation between slices reduces contention and scheduling latency compared to traditional vGPU alone.

A single VM that is attached to a GPU in MIG mode will have access to all MIG slices of that GPU. There is no individual assignment of MIG slices among multiple VMs of a GPU in



[Privacy Policy](#) | [Your Privacy Choices](#) | [Terms of Service](#) | [Accessibility](#) | [Corporate Policies](#) | [Product Security](#) | [Contact](#)

Copyright © 2013-2026, NVIDIA Corporation.

Last updated on Mar 24, 2026.